

APUNTES JAVASCRIPT 2024

ÍNDICE

1. Inserción de Código JavaScript en HTML
2. Tipos de Datos y Variables
 - a) Declaración de variables
 - b) Tipos de datos primitivos
 - c) Tipos de datos objeto
3. Arrays en JavaScript
4. Pila (stack) y Montón (heap) de memoria
5. Iteraciones y Bucles
 - a) Ciclo *for clásico*
 - b) Ciclo *for...of*
 - c) Método *forEach()*
6. Conversión de Tipos y Evaluación en Condiciones
7. Operadores Lógicos y de Comparación
 - a) Operadores Cortocircuito (*&&, ||*)
8. Operador Ternario y Nullish Coalescing (*??*)
9. Métodos y Herramientas de Consola
10. Encadenamiento Opcional (*?.*)
11. Template Strings
12. Spread Operator
13. Trabajar con Módulos
14. NaN: Qué Es y Por Qué Es Importante
15. Desestructuración

1. INSERCIÓN DE CÓDIGO JAVASCRIPT EN HTML

Los bloques de código JavaScript se pueden incluir en un HTML, tanto en **<head>** como en **<body>**.

Si se coloca en **<head>**, el script **se ejecuta antes** y puede retrasar el renderizado del body. También se puede usar el evento **DOMContentLoaded** para asegurarse de que el DOM esté completamente cargado antes de ejecutar el código

Si se coloca en **<body>**, **primero se renderiza el contenido** HTML y después se ejecuta el script.

ASYNC y DEFER:

Estos **atributos**, indican como debe cargarse el archivo JavaScript.

- **ASYNC**: El script se cargará de forma paralela al HTML, es decir, no bloqueará la ejecución de la página.

```
<script async src="script1.js"></script>
```

- **DEFER**: El script se pospondrá hasta que se haya completado el análisis de HTML. Asegura que el script se ejecute en el orden correcto si hay dependencias.

```
<script defer src="script2.js"></script>
```

2. TIPOS DE DATOS Y VARIABLES

2.1 Declaración de variables:

var -> Variable con **alcance global** o de función.

Esta variable **está en desuso** porque no respeta los bloques de código, es decir, sigue siendo accesible fuera del bloque en el que se declara.

Sufre de Hoisting, si se intenta usar la variable antes de ser declarada el resultado será **'undefined'** y no un error. Esto se debe a que la asignación de la variable se eleva hasta el inicio del script o del bloque de código, pero no su valor.

let -> Variable con alcance de bloque, lo que significa que solo es accesible dentro del bloque **{ }** donde se declara.

const -> Constante que no permite cambiar el tipo de dato una vez definido, es inmutable.

2.2 Datos primitivos:

Los datos primitivos son tipos de datos básicos, inmutables y no tienen métodos.

Number -> Representa números, enteros y flotantes (decimales).

String -> Representa cadenas de texto. Se puede usar comillas simples o dobles.

Números grandes o BigInt -> Si un número es demasiado grande, se puede añadir una 'n' al final para convertirlo en un BigInt.

```
let numeroGrande = 5n;
```

Booleanos -> Solo tienen dos valores, 'true' o 'false' (1 o 0).

Si se hace la suma de 'true + true' el resultado será '2'. Esto se debe a que se está sumando su valor numérico.

Null -> Es un tipo de dato que reserva un espacio en memoria pero indica que la variable aún no tiene valor.

Undefined -> Representa una variable que ha sido declarada pero aún no se le ha asignado un valor, por lo tanto no reserva memoria adicional.

Symbol -> Tipo de dato único y anónimo que se usa como identificador

```
let f = Symbol("texto identificativo"); // "f" es un símbolo único
```

2.3 Datos objeto:

Un objeto es una colección de propiedades, donde cada propiedad es una clave (key) que se asocia a un valor. Los valores pueden ser de cualquier tipo, incluido otros objetos.

```
const persona = {
  nombre: "Ana",
  edad: 23,
  tieneDni: true
}; // Este objeto tiene 3 propiedades: nombre, edad, tieneDni seguido de su valor.
```

Acceder a las propiedades de un objeto:

```
console.log(persona.edad); // Siempre se puede acceder con objeto.propiedad
```

```
let a = { id: 4, hash: 'jkl012', prevHas: '0xmnq' };
let clave = "id";
console.log(a[clave]); // Imprime 4
```

Métodos para trabajar con las propiedades de un objeto:

- **Object.keys()** -> Devuelve un array con todas las claves de las propiedades de un objeto.

```
console.log(Object.keys(persona)); // Imprime: ["nombre", "edad", "tieneDni"]
```

- **Object.values()** -> Devuelve un array con todos los valores de las propiedades de un objeto.

```
console.log(Object.values(persona)); // Imprime: ["Ana", 23, true]
```

- **Object.assign()** -> Se usa para copiar los valores de todas las propiedades de uno o mas objetos a un objeto de destino. Es útil para fusionar objetos o crear copias.

```
const destino = { nombre: "Carlos" };
const fuente = { edad: 30 };
```

```
const resultado = Object.assign(destino, fuente); // El primer parámetro es el
destino donde se añadirán los valores del segundo parámetro "fuente"
console.log(resultado); // { nombre: "Carlos", edad: 30 }
```

Ejemplo de una copia superficial:

```
const original = { nombre: "Ana", edad: 23 };
const copia = Object.assign({}, original);
console.log(copia); // { nombre: "Ana", edad: 23 }
```

Object.assign({ }, original): Crea un nuevo objeto vacío ({ }) y copia todas las propiedades del objeto original a este nuevo objeto. Ahora, copia es un nuevo objeto independiente de original, pero comparte los mismos valores en las propiedades.

Ejemplo de comportamiento con objetos anidados:

```
const original = { nombre: "Ana", direccion: { ciudad: "Madrid" } };
const copia = Object.assign({}, original);
copia.direccion.ciudad = "Barcelona";
```

```
console.log(original.direccion.ciudad); // "Barcelona" (¡El objeto original también se
modifica!)
```

Object.assign({ }, original): Crea un nuevo objeto vacío ({ }) y copia todas las propiedades del objeto original a este nuevo objeto. Ahora, copia es un nuevo objeto independiente de original, pero comparte los mismos valores en las propiedades.

Aunque **copia** es un nuevo objeto, la propiedad **direccion es un objeto anidado**, por lo que no se copió su contenido, sino que **se pasó la referencia**. Cuando se modifica `copia.direccion.ciudad`, también se modifica `original.direccion.ciudad` **porque ambos objetos comparten la misma referencia a direccion**.

- **Object.entries()** -> Convierte un objeto en un array de arrays, donde cada array interno tiene un par clave-valor.

```
const persona = {
  nombre: "Ana",
  edad: 23
};
console.log(Object.entries(persona)); // [["nombre", "Ana"], ["edad", 23]]
```

Este método es útil cuando se quiere recorrer tanto las claves como los valores de un objeto al mismo tiempo.

```
for (const [clave, valor] of Object.entries(persona)) {
  console.log(`${clave}: ${valor}`);
}
```

`Object.entries(persona)` convierte el objeto `persona` en un array de pares clave-valor. Con un `for...of` se recorren estos pares. El bucle asigna cada clave a la variable “clave” y cada valor a la variable “valor”, que luego se imprime en la consola.

Casos prácticos:

1. Mostrar detalles de un perfil de usuario

```
const usuario = {
  nombre: "Ana",
  edad: 23,
  ciudad: "Madrid",
};

// Mostrar los datos del usuario de forma organizada
function mostrarPerfil(usuario) {
  console.log("Perfil del usuario:");
  for (const [clave, valor] of Object.entries(usuario)) {
    console.log(`${clave}: ${valor}`);
  }
}

mostrarPerfil(usuario);
```

2. Validar campos de un formulario

```
const formulario = {
  nombre: "Ana",
  edad: "",
  ciudad: "Madrid"
};

// se recorre cada par para verificar si el valor está vacío
function validarFormulario(datos) {
  for (const [campo, valor] of Object.entries(datos)) {
    if (!valor) {
      console.log(`El campo ${campo} está vacío.`);
    }
  }
}

validarFormulario(formulario);
```

- **Object.fromEntries()** -> Convierte un array de pares clave-valor en un objeto.

```
const entries = [["nombre", "Ana"], ["edad", 23]];
const persona = Object.fromEntries(entries);
console.log(persona); // { nombre: "Ana", edad: 23 }
```

Este método es útil para crear un objeto a partir de matrices o arrays.

- **Object.hasOwnProperty()** -> Verifica si un objeto tiene una propiedad con un nombre específico.

```
const persona = { nombre: "Ana" };

console.log(persona.hasOwnProperty("nombre")); // true
console.log(persona.hasOwnProperty("edad")); // false
```

Este método es útil para asegurar que el objeto contiene la propiedad especificada.

Caso práctico:

1. Validar la existencia de propiedades antes de usarlas

```
const persona = {
  nombre: "Ana",
  edad: 23
};
```

```
function verificarPropiedad(obj, propiedad) {
  if (obj.hasOwnProperty(propiedad)) {
    console.log(`La propiedad '${propiedad}' existe: ${obj[propiedad]}`);
  } else {
    console.log(`La propiedad '${propiedad}' no existe en el objeto.`);
  }
}
```

```
verificarPropiedad(persona, "nombre"); // La propiedad 'nombre' existe: Ana
verificarPropiedad(persona, "ciudad"); // La propiedad 'ciudad' no existe en el objeto.
```

2.4 Acceder y modificar propiedades dinámicamente:

Se puede utilizar notación de punto (.) o de corchetes ([]) para acceder a las propiedades de forma dinámica.

```
const a = { n: "a" };

// Acceso y agregación dinámica a una propiedad utilizando corchetes
a["s"] = 2;

console.log(a); // { n: "a", s: 2 }

// Modificación de una propiedad existente utilizando "."
a.n = "few";

console.log(a); // { n: "few", s: 2 }
```

No se pueden añadir variables como claves utilizando el operador punto “.”. Se crearía la clave utilizando literalmente el nombre de la variable.

3. ARRAYS EN JAVASCRIPT

Qué es un array

Es una estructura que permite almacenar múltiples valores en una sola variable. Cada valor en un array tiene una posición (índice), que comienza en 0.

Declaración de arrays

Existen varias formas de crear uno:

- Usando corchetes “[]”

```
const a = [1, 2, 3, [4, 3], 9, 0]; // El array contiene 6 elementos, de los cuales
uno es un array anidado ([4, 3]).
```

- Usando el constructor "Array()"

```
const b = new Array('a', 'b', 'c'); // El array contiene 3 elementos tipo String
```

Características de los arrays

- Pueden contener cualquier tipo, números, strings, objetos u otros arrays.
- Son dinámicos, se puede añadir y eliminar elementos sin importar el tamaño del array.

Acceso a elementos de arrays

Se accede a través de su índice:

```
let valor = a[0]; // valor será 1
```

Métodos comunes para manipular Arrays que **modifican** su valor original:

- **.push()**: Añade un elemento al final del array.

```
let arr = [1, 2, 3];
arr.push(4); // arr ahora es [1, 2, 3, 4]
```

Devuelve: la nueva longitud del array

- **.pop()**: Elimina el último elemento del array.

```
let arr = [1, 2, 3];
arr.pop(); // arr ahora es [1, 2]
```

Devuelve: el elemento eliminado.

- **.shift()**: Elimina el primer elemento del array.

```
let arr = [1, 2, 3];
arr.shift(); // arr ahora es [2, 3]
```

Devuelve: el elemento eliminado.

- **.unshift()**: Añade uno o más elementos al principio del array.

```
let arr = [1, 2, 3];
arr.unshift(0); // arr ahora es [0, 1, 2, 3]
```

Devuelve: la nueva longitud del array después de añadir el/los elemento/s

- **.splice()**: Añade, reemplaza o elimina elementos en una posición específica.

```
let arr = [1, 2, 3];
arr.splice(1, 1, 10); // arr ahora es [1, 10, 3] (elimina 1 elemento en índice 1 y
añade 10)
```

Devuelve: un array con los elementos eliminados o un array vacío si no se elimina nada.

- **.sort()**: Ordena los elementos del array.

```
let arr = [3, 1, 2];
```

```
arr.sort(); // arr ahora es [1, 2, 3]
```

Devuelve: el array modificado, ya ordenado.

- **.reverse()**: Invierte el orden de los elementos del array.

```
let arr = [1, 2, 3];
```

```
arr.reverse(); // arr ahora es [3, 2, 1]
```

Devuelve: el array modificado, ya invertido.

- **.fill(valor, inicio, fin)**: Rellena todos los elementos del array.

```
let arr = [1, 2, 3];
```

```
arr.fill(0); // arr ahora es [0, 0, 0]
```

```
let arr = [1, 2, 3, 4, 5];
```

```
arr.fill(0, 1); // Comienza a rellenar con 0 desde el índice 1
```

```
console.log(arr); // Resultado: [1, 0, 0, 0, 0]
```

```
arr.fill(0, 1, 3); // Rellena con 0 desde el índice 1 hasta el 3 (sin incluir el 3)
```

```
console.log(arr); // Resultado: [1, 0, 0, 4, 5]
```

Devuelve: el array modificado con los nuevos valores

- **.copyWithin(destino, inicio, fin)**: Copia una parte del array dentro del mismo, sin modificar su longitud.

```
let arr = [1, 2, 3, 4];
```

```
arr.copyWithin(1, 2); // arr ahora es [1, 3, 4, 4]
```

```
let arr = [1, 2, 3, 4, 5];
```

```
arr.copyWithin(2); // Copia desde el índice 0 hasta el final y pega desde el índice 2
```

```
console.log(arr); // Resultado: [1, 2, 1, 2, 3]
```

```
let arr = [1, 2, 3, 4, 5];
```

```
arr.copyWithin(0, 2, 4); // Copia desde el índice 2 hasta el 4 (sin incluir) y lo pega desde el índice 0
```

```
console.log(arr); // Resultado: [3, 4, 3, 4, 5]
```

Devuelve: el array modificado con los elementos copiados y sobrescritos.

Métodos comunes para manipular Arrays que **NO modifican** su valor original:

- **.slice()**: Extrae una sección del array y lo copia.

```
let numeros = [1, 2, 3, 4, 5];
let resultado = numeros.slice(1, 4);
console.log(resultado); // [2, 3, 4] (nueva porción del array)
console.log(numeros); // [1, 2, 3, 4, 5] (array original sin cambios)
```

Devuelve: Un nuevo array con los elementos seleccionados.

- **.concat()**: Combina dos o más arrays en uno solo.

```
let arr1 = [1, 2];
let arr2 = [3, 4];
let resultado = arr1.concat(arr2);
console.log(resultado); // [1, 2, 3, 4]
console.log(arr1); // [1, 2] (sin cambios)
console.log(arr2); // [3, 4] (sin cambios)
```

Devuelve: Un nuevo array con la combinación de los arrays.

- **.indexOf()**: Busca el índice de la primera aparición de un elemento en un array o cadena de texto.

```
const numeros = [10, 20, 30, 40, 50];
console.log(numeros.indexOf(30)); // 2 (el índice donde se encuentra 30)
console.log(numeros.indexOf(100)); // -1 (no se encuentra el número 100)
```

Otro ejemplo:

```
const aoCollect = [5,9,8,1,5]
export const eliminarElemento = (aoCollect = [], asElement) => {
  let lnPos = aoCollect.indexOf(asElement);
  if (lnPos !== -1){
    aoCollect.splice(lnPos, 1); // Elimina el elemento del array
  }
  return aoCollect
};
```

Devuelve: La posición de la primera aparición del elemento.

- **.findIndex()**: Busca un elemento en el array basado en una condición.

```
let numeros = [5, 12, 8, 130, 44];
let resultado = numeros.findIndex(num => num > 10);
console.log(resultado); // 1 (el índice del primer número mayor que 10)
```

Devuelve: El índice (posición) del primer elemento que cumple con la condición o -1 si no encuentra ninguno.

- **.filter()***: Crea un nuevo array con todos los elementos que cumplan la condición.

```
let numeros = [5, 12, 8, 130, 44];  
let resultado = numeros.filter(num => num > 10);  
console.log(resultado); // [12, 130, 44]  
console.log(numeros); // [5, 12, 8, 130, 44] (sin cambios)
```

Devuelve: Un nuevo array con los elementos que cumplen la condición.
Es interesante cuando se quiere eliminar elementos de un array, ya que crearía un nuevo array sin los elementos que coinciden con la condición.

- **.reduce()***: Ejecuta una función reductora sobre cada elemento del array acumulando un solo valor.

```
let numeros = [1, 2, 3, 4];  
let resultado = numeros.reduce((acumulador, valorActual) => acumulador +  
valorActual, 0);  
console.log(resultado); // 10 (la suma de todos los elementos)  
console.log(numeros); // [1, 2, 3, 4] (sin cambios)
```

Devuelve: El valor acumulado.

- **.some()**: Comprueba si al menos un elemento del array cumple una condición.

```
let numeros = [1, 2, 3, 4];  
let resultado = numeros.some(num => num > 2);  
console.log(resultado); // true (porque al menos un número es mayor que 2)
```

Devuelve: true si encuentra algún elemento, false si no.

- **.every()**: Comprueba si todos los elementos del array cumplen una condición.

```
let numeros = [1, 2, 3, 4];  
let resultado = numeros.every(num => num > 0);  
console.log(resultado); // true (todos los números son mayores que 0)
```

Devuelve: true todos cumplen la condición, false si no.

- **.map()***: Aplica una función a cada elemento del array.

```
let numeros = [1, 2, 3, 4];  
let resultado = numeros.map(num => num * 2);  
console.log(resultado); // [2, 4, 6, 8] (nuevo array)  
console.log(numeros); // [1, 2, 3, 4] (sin cambios)
```

Devuelve: un nuevo array con los elementos transformados.

- **.reduce()***: Reduce un array a un solo valor, aplicando una función a cada uno de los elementos y acumulando un resultado.

```
const numeros = [1, 2, 3, 4];
const suma = numeros.reduce((acumulador, valorActual) => acumulador +
valorActual, 0); // 0 valor inicial del ac. Por defecto es el 1er valor del array.
console.log(suma); // Imprime: 10
```

Devuelve: un único valor, resultado de la operación.

Otro ejemplo de .reduce():

```
const filtrarPares = (aoArr = []) => {
  return aoArr.reduce((acumulador, num) => {
    if(num % 2 === 0){
      acumulador.push(num);
    }
  }, [])
};
```

Forma reducida:

```
const filtrarPares = (aoArr = []) => {
  return aoArr.reduce((acumulador, num) =>
    (num % 2 === 0 ? acumulador.push(num) : null, acumulador), []);
};
```

Otro ejemplo de .reduce():

```
const contarPalabras = (asCadena = "") => {
  const lsWords = asCadena.split(" ");

  return lsWords.reduce((acumulador, actual) =>
    (acumulador[actual] = (acumulador[actual] || 0) + 1, acumulador),
    {}
  );
};
```

Estructuras de datos avanzadas:

- **Set:** Estructura que permite almacenar valores únicos, es decir, no permite duplicados.

Características:

- No permite duplicados.
- Puede almacenar cualquier tipo de valor.
- Los elementos dentro de un Set no tienen un orden particular.

Métodos principales de un Set:

.add(valor): Añade un valor al Set.

.delete(valor): Elimina un valor del Set.

.has(valor): Verifica si un valor existe en el Set.

.clear(): Elimina todos los elementos del Set.

.size: Devuelve el número de elementos en el Set.

Ejemplos de uso:

```
const miSet = new Set([1, 2, 3, 3]); // Crea un set con 3 elementos únicos
console.log(miSet); // Set {1, 2, 3}
```

```
miSet.add(4); // Añade el número 4 al Set
```

```
miSet.delete(2); // Elimina el número 2 del Set
```

```
console.log(miSet.has(1)); // Verifica si el 1 está en el Set (true)
```

```
console.log(miSet.size); // Muestra el tamaño del Set (3)
```

Operaciones comunes:**Unión**

```
const setA = new Set([1, 2, 3]);
```

```
const setB = new Set([3, 4, 5]);
```

```
const union = new Set([...setA, ...setB]); // Set {1, 2, 3, 4, 5}
```

Intersección

```
const interseccion = new Set([...setA].filter(x => setB.has(x))); // Set {3}
```

Diferencia

```
const diferencia = new Set([...setA].filter(x => !setB.has(x))); // Set {1, 2}
```

4. PILA (STACK) Y MONTÓN (HEAP) DE MEMORIA:

Éstas son dos áreas donde se almacenan los datos de un programa.

Pila: Los datos se manejan de manera rápida y directa aún que con algunas limitaciones. Es una estructura de tipo **LIFO** (último en entrar, primero en salir). *Aquí se almacenan los primitivos* por ser simples y de tamaño fijo.

Algunas características son:

- Tamaño limitado
- Acceso rápido
- Almacenamiento temporal

Montón: Aquí se almacenan los datos mas complejos o dinámicos (arrays, objetos, funciones...). No mantiene una estructura, por lo que los datos se almacenan en cualquier parte del montón y se gestionan por referencias o punteros.

Algunas características son:

- Tamaño variables
- Memoria no estructurada
- Asignación dinámica
- Recolección de basura (los objetos se eliminan cuando no tienen referencias).

5. ITERACIONES Y BUCLES

Ciclo for clásico

Es una de las estructuras de control más usadas para iterar sobre elementos de un array o ejecutar un bloque de código un número específico de veces.

```
for (let i = 0; i <= 10; i++) {
    console.log(i); // Se imprimirá el valor de i 10 veces
}
```

También es útil para recorrer arrays (aún que no es el preferido):

```
const frutas = ["manzana", "banano", "uva"];

for (let i = 0; i < frutas.length; i++) {
    console.log(`Índice ${i}: ${frutas[i]}`);
}
```

Ciclo for...of

Es una forma sencilla de iterar sobre objetos iterables como arrays, cadenas, mapas...etc. Cada vez que se ejecuta, toma el siguiente valor del iterable y lo asigna a la variable declarada.

```
const nombres = ["Ana", "Luis", "María"]; // Se puede utilizar const ya que la variable
se redefine en cada ciclo y no cambia durante la iteración.

for (const nombre of nombres) { // Se aconseja usar el singular de la variable
    console.log(nombre);
}
```

Este ciclo es útil cuando solo interesa el valor de cada elemento y no su índice.

Método `forEach()`

Es un método de los arrays, los recorre y ejecuta una función para cada elemento del array y es muy común verlo con arrow functions (funciones flecha), para simplificar la sintaxis.

- No modifica el array original a menos que se acceda y modifique implícitamente.
- No devuelve un nuevo array ni retorna ningún valor, su único propósito es ejecutar una función para cada elemento.

La sintaxis extendida de este método es:

```
array.forEach((elemento, indice, arrCompleto) => { //Solo es obligatorio "elemento".
  console.log(` ${elemento}, Índice: ${indice}, Array completo: ${arrCompleto} `);
})
```

Resumen de parámetros en `forEach()`:

- **Primer parámetro** (obligatorio): Valor del elemento actual que se itera en el array.
- **Segundo parámetro** (opcional): Índice del elemento dentro del array.
- **Tercer parámetro** (opcional): Array completo que se esta iterando.

Otro ejemplo con el elemento únicamente como parámetro:

```
const nombres = ["Ana", "Luis", "María"];
nombres.forEach(nombre => console.log(nombre)); // Imprime cada elemento.
```

Si la función solo tiene un parámetro, no es necesario usar paréntesis. Si solo tiene una instrucción también se pueden omitir las “{ }” y la palabra “return”

Ejemplo más completo de arrow function en `forEach()`:

```
const estudiantes = [
  { nombre: "Ana", nota: 8 },
  { nombre: "Pepe", nota: 5 },
  { nombre: "María", nota: 9 }
];
estudiantes.forEach((estudiante, indice) => {
  if (estudiante.nota >= 6) {
    console.log(`${indice}: ${estudiante.nombre} ha aprobado.`);
  } else {
    console.log(`${indice}: ${estudiante.nombre} ha reprobado.`);
  }
});
```

Ejemplo para modificar el array original con forEach():

```
const numeros = [1, 2, 3];

numeros.forEach((numero, indice, arr) => {
    arr[indice] = numero * 2; // Modificamos el array original accediendo a la
    posición "indice" del array pasado por argumentos "arr".
});

console.log(numeros); // [2, 4, 6]
```

6. CONVERSIÓN DE TIPOS Y EVALUACIÓN EN CONDICIONES**Conversión de String a Number:**

Para convertir un String en número, se coloca un signo "+" delante de la variable:

```
let a = "2";
+a // a = 2
```

Conversión implícita en Condiciones:

Los valores en Condiciones como "if()" se convierten automáticamente a booleanos:

```
let numero = 5;
if (numero) { // Evalúa si existe y además si no es un valor "falsy"
    console.log("Entra aquí");
}
```

Convertir cualquier valor a booleano usando "!!"

El operador doble negación (!!) se utiliza para convertir cualquier valor en su equivalente booleano. Esto es útil en situaciones donde se necesita asegurar que el valor sea un booleano, como dentro de un bloque if:

```
let valor = "hola";
console.log(!!valor); // true, porque "hola" es un valor truthy
```


TRUTHY / FALSY

Estos términos se utilizan en JavaScript (entre otros) para describir cómo los valores son interpretados en un contexto booleano, como una condición “if()”.

Truthy: Cualquier valor que, aún que no sea literalmente “true”, JavaScript lo trata como tal.

Algunos valores truthy son:

- true
- Cualquier número distinto de 0.
- Cadenas no vacías (“Hola”...).
- Infinity
- Arrays y objetos no vacíos ([], { }) *Esto se debe a que se les asigna una ubicación en memoria aún en vacío ya que no se evalúan por su contenido si no por su referencia.*

Falsy: De igual modo, se trata como un “false”.

Algunos valores falsy son:

- false
- 0 (número 0)
- “ ” (cadena vacía)
- null
- undefined
- NaN (Not-a-Number)

7. OPERADORES LÓGICOS Y DE COMPARACIÓN

AND (&&): devuelve ‘true’ si ambos operandos son verdaderos.

OR (||): devuelve true solo si al menos uno de los operandos es verdadero.

==: compara valores sin tener en cuenta el **tipo** de dato.

console.log(5 == '5'); // true, no tiene en cuenta que uno es tipo number y otro String

=== (igualdad estricta): compara valores teniendo en cuenta tanto el valor como el tipo de dato.

console.log(5 === '5'); // false, el valor coincide pero el tipo no.

Operadores comunes:

!= Desigualdad (sin tipo).

!== Desigualdad estricta (con tipo).

> Mayor que.

< Menor que.

>= Mayor o igual que.

<= Menor o igual que.

Operadores de Cortocircuito

Son una manera eficaz de optimizar las evaluaciones lógicas, ya que detienen la ejecución tan pronto como se encuentra un valor que determina el resultado final de la expresión.

Operadores principales:

- AND (&&) y OR (||)

AND lógico (&&) -> Si el primer valor es “falsy”, la expresión retorna ese valor y se detiene. Si todos los valores son “truthy”, devuelve el último.

```
let c = 1;
let g = 2;
let j = 0;
let f = 1;
let k = 8;
let b = c && g && j && f || k;
```

La evaluación se realiza por tramos de izquierda a derecha, es decir:

Primero se evalúa:

```
c && g && j && f
```

Si alguno (c, g, j o f) son “falsy”, para de evaluar y pasa al siguiente tramo:

```
c = 1, es truthy, continúa la evaluación.
g = 2, es truthy, continúa la evaluación.
j = 0, es falsy, se detiene la evaluación sin importar el valor de f.
b = j || 0
```

Segundo tramo de evaluación:

```
b = j || k
```

Se retorna el primer valor truthy:

*j = 0, es falsy, se evalúa k.
k = 8, es truthy, por lo tanto b = 8;*

Si j y f hubiesen sido truthy, k no se evalúa y b = f. Si por el contrario, todos hubiesen sido falsy, b = k; porque se retorna el primer valor truthy que aparece, al no haber ninguno la evaluación continua hasta el final.

OR lógico (||) -> Si el primer valor es “truthy”, la expresión retorna ese valor y se detiene. Si es “falsy”, continúa evaluando el siguiente valor. Si todos los valores son “falsy”, devuelve el último.

let b = null || undefined || 0 && 9;

Primero se evalúa:

null || undefined || 0

Los tres son falsy, por lo tanto se devuelve el último valor, b = 0.

La siguiente evaluación sería:

0 && 9

El operador && solo se evalúa si el valor de la izquierda es truthy, como 0 es falsy, esta parte no se evalúa y el resultado final es 0.

La principal diferencia con “??”, es que éste considera solo null y undefined como valores “falsy” .

8. OPERADOR TERNARIO ‘?’ Y NULLISH COALESCING OPERATOR ‘??’

Forma abreviada de representar una estructura if...else. Evalúa una expresión y devuelve un valor dependiendo de si la condición es ‘true’ o ‘false’.

*let edad = 18;
let resultado = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
console.log(resultado); // "Mayor de edad"*

“??” -> Este operador devuelve el operando de la derecha si el operando de la izquierda es null o undefined. Es útil para establecer valores predeterminados.

*let nombre = null;
let resultado = nombre ?? "Usuario Anónimo";
console.log(resultado); // "Usuario Anónimo"*

9. MÉTODOS Y HERRAMIENTAS DE CONSOLA

console.log() -> Se usa para imprimir un mensaje o el valor de una variable en la consola del navegador.

console.error() -> Muestra errores en la consola.

console.table() -> Muestra datos en forma de tabla.

Typeof -> Se usa para conocer el tipo de dato de una variable.

10. ENCADENAMIENTO OPCIONAL (?.)

Nueva característica de JavaScript que permite acceder a las propiedades de un objeto de forma segura, evitando errores cuando la propiedad que se intenta acceder no existe o es null o undefined.

```
const transacciones = [{ mont: 100 }, { mont: 200 }];
```

```
console.log(transacciones?.[0]?.mont); // 100
```

```
console.log(transacciones?.[1]?.mont); // 200
```

```
console.log(transacciones?.[2]?.mont); // undefined (porque el índice 2 no existe)
```

11. TEMPLATE STRINGS

Los template strings permiten incluir variables dentro de una cadena de texto de manera sencilla usando comillas francesas/invertidas (```) y (`${ }`) para insertar variables.

```
let nombre = "Ana";
```

```
console.log(`Mi nombre es ${nombre}`); // Imprime: Mi nombre es Ana
```

12. SPREAD OPERATOR

`...` -> Este operador permite “expandir” elementos de un array o cadena, en sus componentes individuales.

```
let z = "hola";
```

```
[...z]; // ["h", "o", "l", "a"]. Convierte 'hola' en un array con cada una de sus letras.
```

También es posible realizar una copia de un array de la siguiente manera:

```
[...w]; // esto daría lugar a ["a", "b", 2, null].
```

Un nuevo array con los mismos valores pero con un valor distinto en memoria.

Los arrays en JavaScript pueden tomar valores de distintos tipos:

```
let w = ["a", "b", 2, null];
```

Fusión de arrays

```
let arr1 = [1, 2, 3];  
let arr2 = [4, 5, 6];  
let fusion = [...arr1, ...arr2]; //[1, 2, 3, 4, 5, 6]
```

13. TRABAJAR CON MÓDULOS

Los módulos permiten organizar y dividir el código en archivos más pequeños y reutilizables, mejorando la estructura y mantenibilidad del código.

Declaraciones export e import

Export: Permite marcar las partes del código que estarán disponibles para otros módulos.

Import: Permite insertar código de otros módulos.

Declarar un archivo JavaScript como módulo

Para que el navegador interprete el archivo como un módulo, se debe declarar en el HTML con el atributo `type="module"`. Sin esto, no se pueden usar las características como “export” e “import”.

```
<script type="module" src="script.js" defer></script>
```

Exportar una variable

```
export const nombre = "Ana";
```

Exportar una función

```
export function saludar() {  
  console.log(`Hola, ${nombre}`);  
}
```

Importar la variable y la función

```
import { nombre, saludar } from './script.js';
```

14. NaN, QUÉ ES Y POR QUÉ ES IMPORTANTE

NaN significa Not-a-Number. Se genera cuando una operación matemática o de conversión falla y el resultado no puede representarse como un número válido.

```
console.log(0 / 0); // NaN
```

La función **isNaN(valor)**, intenta convertir el valor a un número antes de verificar si es NaN, por lo tanto primero trata de convertirlo implícitamente a un número y si esa conversión falla, devuelve 'true' porque el resultado es NaN.

La función **Number.isNaN(valor)**, no intenta realizar ninguna conversión, directamente comprueba si el valor es exactamente NaN. Ésta es una forma más estricta y precisa de comprobar si un valor es realmente NaN.

Valor	isNaN(valor)	Number.isNaN(valor)
"texto"	true	false
undefined	true	false
NaN	true	true
12	false	false
"true"	false	false

15. DESESTRUCTURACIÓN

Permite extraer valores de arrays o propiedades de objetos en variables separadas de una forma muy concisa y legible.

Desestructuración de arrays

Cuando se desestructura un array, se puede asignar sus valores a variables individuales, según sus posiciones.

```
const numeros = [10, 20, 30];
```

```
// Desestructuración completa
```

```
const [primero, segundo, tercero] = numeros;
```

```
console.log(primero); // 10
```

```
console.log(segundo); // 20
```

```
console.log(tercero); // 30
```

```
// Desestructuración parcial
```

```
const numeros = [10, 20, 30];
```

```
// Solo extraemos el primero y el tercero
```

```
const [primero, , tercero] = numeros;
```

```
console.log(primero); // 10
```

```
console.log(tercero); // 30
```

Desestructuración de objetos

Con los objetos, la desestructuración permite extraer propiedades por el nombre.

```
const persona = {  
  nombre: "Ana",  
  edad: 25,  
  ciudad: "Madrid"  
};
```

// Desestructuración

```
const { nombre, edad, ciudad } = persona;
```

```
console.log(nombre); // "Ana"  
console.log(edad);   // 25  
console.log(ciudad); // "Madrid"
```

En este caso, las variables nombre, edad y ciudad, toman los valores correspondientes de las propiedades del objeto persona.

Cambiar el nombre de las variables

Se puede asignar un nombre de variable diferente al de la propiedad del objeto:

```
const persona = {  
  nombre: "Ana",  
  edad: 25  
};
```

// Renombrando durante la desestructuración

```
const { nombre: nombrePersona, edad: edadPersona } = persona;
```

```
console.log(nombrePersona); // "Ana"  
console.log(edadPersona);   // 25
```

Aquí, persona es el nombre del objeto que contiene las propiedades nombre y edad.

La sintaxis `const { nombre, edad = 30 } = persona;` le dice a JavaScript: "del objeto llamado persona, extrae las propiedades nombre y edad, y asigna esos valores a variables con el nombre nombrePersona y edadPersona."

En el caso de recibir un objeto en una función, se puede desestructurar sin necesidad de hacer referencia al nombre del objeto.

```
function mostrarDatos({ nombre, edad }) {
  console.log(`Nombre: ${nombre}, Edad: ${edad}`);
}
```

```
mostrarDatos({ nombre: "Carlos", edad: 30 });
```

Asignar valores por defecto

Se puede asignar un valor por defecto si la posición o propiedad no existe.

```
const colores = ["rojo", "azul"];

// Tercera posición no existe, le damos un valor por defecto
const [primero, segundo, tercero = "verde"] = colores;

console.log(tercero); // "verde"
```

```
const persona = { nombre: "Ana" };

// Si 'edad' no existe, le damos un valor por defecto de 30
const { nombre, edad = 30 } = persona;

console.log(edad); // 30
```

Desestructuración anidada

```
const empleado = {
  nombre: "Luis",
  detalles: {
    edad: 28,
    departamento: "Marketing"
  }
};
```

// Desestructuración anidada

```
const { nombre, detalles: { edad, departamento } } = empleado;
```

```
console.log(edad); // 28
console.log(departamento); // "Marketing"
```

Ejemplo de uso común en funciones

```
const ofuscarArray = (aoArray = []) => {
  return aoArray
    .map(value => ({ value, sort: Math.random() }))
    .sort((a, b) => a.sort - b.sort)
    .map(({ value }) => value); // Uso de desestructuración para extraer los valores.
}
```


Explicación del código:

1. Se crean objetos “clave : valor”, tomando como clave el valor original y como valor el número aleatorio (llamado sort).

```
aoArray.map(value => ({ value, sort: Math.random() }));
```

El resultado de esto, sería algo así:

Cada número original, está emparejado con un número aleatorio.

```
[
  { value: 1, sort: 0.76 },
  { value: 2, sort: 0.23 },
  { value: 3, sort: 0.45 },
  { value: 5, sort: 0.12 }
]
```

2. Se ordena según el número aleatorio “sort”

Como el objetivo del ejemplo es ofuscar el array, se ordenan los objetos según el valor aleatorio que tiene asignado cada valor original.

```
.sort((a, b) => a.sort - b.sort);
```

Se emplea una función que compara dos elementos a la vez, donde a y b toman valores del objeto como por ejemplo:

```
a = { value: 2, sort: 0.23 }
b = { value: 3, sort: 0.45 }
```

Si el resultado de $a - b$ es positivo, a irá detrás de b, por el contrario si es negativo a irá antes que b.

$$0.23 - 0.45 = -0.22$$

Después de ordenar los valores, el objeto ahora se vería así:

```
[
  { value: 5, sort: 0.12 },
  { value: 2, sort: 0.23 },
  { value: 3, sort: 0.45 },
  { value: 1, sort: 0.76 },
]
```

3. Extraer solo los valores “value”, utilizando desestructuración:

Ya están todos los valores mezclados, que es el objetivo del ejemplo. Pero el resultado muestra además los valores aleatorios que se han generado, para evitarlo, se emplea otro `.map( )` que servirá para tomar solo la propiedad “value” de cada objeto.

```
.map(({ value }) => value);
```

`{ value }` -> equivale a acceder a las propiedades de forma habitual, como:

```
array.map(obj => {  
  const value = obj.value; // Esto es lo que hace la desestructuración por ti  
  return value;  
});
```